

AI STUDY PLANNER

Multi-Step Generative AI Workflow for Personalized Study Planning & Learning Support

STUDENT	G. SAINATH REDDY
REGISTER NO.	99230040546
DATE	06-09-2026
ASSESSMENT	Mini Assessment — GenAI Workflow Design

01 | Concept

Purpose: The AI Study Planner is a Generative AI application that creates personalized study plans based on a student's subjects, exam dates, available study hours, preferred study times, difficult topics, and learning goals. The system uses an LLM to organize study tasks into a practical daily or weekly schedule.

Problem: Students often struggle to divide their available time between multiple subjects and topics. A fixed timetable may not consider exam priorities, difficult subjects, available hours, or changes in the student's schedule. A GenAI-based planner can dynamically create and revise a study plan.

02 | Objective

The workflow uses sequential prompts and Generative AI to understand student requirements, prioritize subjects, allocate study time, generate a structured timetable, and provide revision tasks and adjustments when the student's available time changes.

03 | Workflow Overview

1. Input	2. Analyze	3. Prioritize	4. Generate	5. Adapt
Student Data	Subjects	Importance	Study Plan	Revision

04 | Workflow Steps

- 1. Student Input & Requirement Collection:** Collect subjects, exam dates, available study hours, preferred study times, difficult subjects, and target goals.
- 2. Study Requirement Analysis:** Analyze subjects and identify which topics require more attention based on exam proximity, difficulty, and stated priorities.
- 3. Priority & Time Allocation:** Generate a reasonable distribution of available study time across subjects while reserving time for revision and practice.
- 4. Study Plan Generation:** Create a daily or weekly timetable containing subjects, topics, study duration, breaks, revision sessions, and practice tasks.
- 5. Adaptive Revision:** When available hours or progress changes, regenerate the remaining schedule while keeping important subjects and revision activities prioritized.

05 | Prompt Workflow

Step 1 — Collect Student Information

Read subjects, exam dates, available hours, preferred schedule, difficult topics, and learning goals.

Step 2 — Analyze Study Requirements

Identify subjects requiring higher priority and organize workload according to available time.

Step 3 — Allocate Study Time

Create a balanced distribution of study hours and include breaks, revision, and practice sessions.

Step 4 — Generate Personalized Plan

Use a structured prompt to generate a clear daily or weekly study timetable with subjects, topics, duration, and goals.

Step 5 — Review & Adapt Plan

Check whether the plan fits available time and regenerate the remaining schedule when requirements change.

06 | Prompt Chaining Architecture

STAGE	INPUT / CONTEXT	OUTPUT
1	Subjects + exam dates + available hours	Student study profile
2	Study profile + priorities	Subject priority list
3	Priority list + available time	Time allocation
4	Allocation + preferences	Daily / weekly study plan
5	Plan + progress / schedule changes	Updated study plan
6	Completed + remaining topics	Revision & practice plan

07 | GenAI Components

- Prompt Templates — keep planning instructions consistent while inserting dynamic student information.
- LLM — understands requirements and generates personalized schedules and recommendations.
- Structured Output — represents sessions using date, subject, topic, duration, and task fields.
- Prompt Chaining — connects requirement analysis, prioritization, planning, and adaptation stages.
- Prompt Evaluation — compares prompt versions using a fixed set of student planning scenarios.

08 | Demonstration Input

Example Scenario: A student has four subjects—Data Structures, DBMS, Operating Systems, and Machine Learning. The student has 4 hours per day available for study and wants extra attention for difficult subjects. The nearest examination is in two weeks.

09 | Expected Final Output & Workflow Result

The system analyzes the subjects and available time, prioritizes the nearest and difficult subjects, creates a balanced daily schedule, includes revision and practice periods, and can regenerate the remaining schedule if the student's available study time changes.

INPUT	4 subjects + 4 hours/day + exam dates + difficult topics
ANALYSIS	Identify priority subjects and available study capacity
ALLOCATION	Distribute study time across subjects + revision
GENAI OUTPUT	Personalized daily/weekly study timetable
ADAPTATION	Regenerate remaining plan when schedule changes

10 | Sample Support Execution Plan

PHASE	ACTION FOCUS	OUTPUT / METRIC
Phase 1	Requirement Collection	Student details → study profile
Phase 2	Priority Analysis	Subjects → priority levels
Phase 3	Time Allocation	Available hours → balanced distribution
Phase 4	Plan Generation	Inputs → daily/weekly timetable
Phase 5	Plan Review	Timetable → schedule consistency check
Phase 6	Adaptive Planning	Progress/changes → revised timetable

11 | Key Benefits

- ✓ Creates personalized study schedules from simple student inputs
- ✓ Automatically prioritizes subjects and difficult topics
- ✓ Makes better use of limited study time
- ✓ Includes revision, practice, and break periods
- ✓ Can adapt the schedule when study time or progress changes
- ✓ Demonstrates practical Generative AI, prompt engineering, and prompt chaining

12 | Discussion Questions & Answers

1. What is the advantage of using {user_input} instead of hard-coding input?

Templating separates fixed planning instructions from dynamic student information. The same prompt can create plans for different students, subjects, exam dates, and available study hours.

2. When would .format() be enough?

Python's .format() is sufficient when a few values such as subject names, exam dates, or available hours need to be inserted into a fixed instruction.

3. When might Jinja2 be useful?

Jinja2 is useful when the prompt structure changes based on conditions, such as different instructions for fewer study hours, multiple exams, or extra revision.

4. Why should prompts be versioned?

Prompt versioning provides an audit trail. If a new prompt creates poor or unbalanced study plans, earlier versions can be compared and restored.

5. What is a prompt regression?

A prompt regression occurs when a prompt change improves one planning scenario but unintentionally makes another scenario worse.

6. Why use the same evaluation set when comparing prompt versions?

A fixed evaluation set makes prompt comparison fair. The same student scenarios allow the developer to measure whether a prompt change improves planning quality.

13 | Suggested Technology Stack

LAYER	TECHNOLOGY OPTIONS
Language	Python
LLM	OpenAI / Gemini / other course-approved LLM
Prompt Framework	LangChain or direct API integration
Interface	Streamlit
Data Storage	SQLite / JSON
Output	HTML / PDF / downloadable timetable
Evaluation	Fixed planning scenarios + rubric-based evaluation